

Federated Learning Decentralizzato Peer-to-Peer con Push One-Hop a Fanout Fisso su FEMNIST

Flavio Di Palma, Giordano Spirito
Corso di Laurea Magistrale in Ingegneria Informatica
Università degli Studi di Roma “Tor Vergata”
Roma, Italia

Abstract—Questo lavoro presenta la progettazione e la valutazione sperimentale di un sistema di Federated Learning decentralizzato con protocollo di comunicazione push one-hop a fanout fisso. Il sistema è applicato al riconoscimento di caratteri manoscritti sul dataset non-i.i.d. FEMNIST del progetto LEAF. A differenza del FedAvg centralizzato, nessun nodo agisce da aggregatore per l'intera durata dell'addestramento: ciascun worker alterna fasi di training locale e di aggregazione dei pesi ricevuti dai vicini, mentre un componente centralizzato leggero è impiegato solo per la service discovery. Il sistema è stato valutato con 16 configurazioni sperimentali (8 in esecuzione locale, 8 su istanze AWS EC2), variando il numero di client (3, 5, 8), il fanout di comunicazione e il numero di passi locali tra due round di comunicazione. I risultati mostrano un'accuratezza di validazione tra l'87% e il 91%, un costo di comunicazione per round di circa 6,8 MB per messaggio e una netta separazione tra il costo del training locale e quello della fase di comunicazione, anche su rete reale.

I. INTRODUZIONE

Il Federated Learning permette a un insieme di nodi client, ciascuno in possesso di un dataset locale privato, di addestrare collaborativamente un modello condiviso senza scambiare i dati grezzi. Nella formulazione centralizzata, un server raccoglie i pesi locali di ogni client, li aggrega secondo un particolare algoritmo e ridistribuisce il modello aggiornato: il server è un single point of failure, la sua banda è un collo di bottiglia che cresce con il numero di client, e ogni round richiede una sincronizzazione globale al passo del client più lento.

Il presente lavoro affronta questi limiti progettando un sistema di FL *decentralizzato*: i pesi vengono scambiati tra i client tramite un protocollo di comunicazione peer-to-peer a push one-hop e fanout fisso, in cui ciascun client invia periodicamente i propri pesi a un sottoinsieme di vicini scelti casualmente, e vengono aggregati localmente da chi li riceve tramite FedAvg. Un componente centralizzato, il discovery server, resta limitato esclusivamente alla registrazione e alla scoperta reciproca dei client.

II. CONTESTO

A. FedAvg

FedAvg aggiorna il modello globale come media dei pesi locali, pesata per il numero di campioni di ciascun client:

$$w \leftarrow \sum_{i=1}^K \frac{n_i}{n} w_i, \quad n = \sum_{i=1}^K n_i \quad (1)$$

dove w_i e n_i sono rispettivamente i pesi e il numero di campioni di addestramento del client i . Sotto ipotesi di dati non-i.i.d., la velocità di convergenza di FedAvg degrada con il grado di eterogeneità tra le distribuzioni locali.

B. DiLoCo e comunicazione sparsa

DiLoCo [1] riduce la frequenza di comunicazione alternando H passi di ottimizzazione locale (*inner steps*) a un singolo passo di sincronizzazione globale, eseguito da un *outer optimizer* con momentum mantenuto centralmente. Il sistema qui presentato adotta la stessa idea di comunicazione sparsa a H passi, ma non può adottare l'outer optimizer di DiLoCo così com'è: quest'ultimo richiede uno stato di momentum globale coerente per tutti i client, il che presuppone un coordinatore centrale. Il sistema sostituisce quindi la sincronizzazione globale periodica di DiLoCo con un'aggregazione locale asincrona basata su un push one-hop a fanout fisso tra vicini, descritta nella Sezione III e motivata per contrasto con le alternative del corso nella Sezione IIE.

C. Dataset: LEAF e FEMNIST

FEMNIST è un dataset di caratteri manoscritti (cifre e lettere, 62 classi, immagini 28×28 in scala di grigi) partizionato per scrittore: ogni scrittore produce una distribuzione di stili ed etichette propria, rendendo il dataset naturalmente non-i.i.d. tra client quando ogni partizione corrisponde a scrittori diversi. Nel sistema qui presentato ogni client riceve dati di *più* scrittori LEAF, per simulare uno scenario realistico in cui un partecipante rappresenta un piccolo gruppo di utenti piuttosto che un singolo individuo.

D. Strategie alternative non implementate

La letteratura propone diverse estensioni di FedAvg per mitigare l'eterogeneità dei dati: FedProx [2], SCAFFOLD [3], FedBN [4]. Il sistema qui presentato implementa esclusivamente FedAvg, scelto come strategia di riferimento più semplice da rendere verificabile in un contesto decentralizzato puro. Non tutte le alternative sono equivalenti dal punto di vista architetturale: FedProx modifica solo la funzione di costo usata in Fase B (Sezione IVB) e potrebbe essere adottata senza alcuna modifica al buffer di aggregazione o al protocollo gRPC; SCAFFOLD richiederebbe di trasmettere anche variabili di controllo oltre ai pesi, aumentando la dimensione dei messaggi ma senza introdurre un aggregatore centrale.

E. Scelta del protocollo di comunicazione

Il corso di riferimento presenta tre famiglie di disseminazione a livello applicativo su overlay non strutturato: *flooding*, *random walk* e *gossip epidemico*, a sua volta nelle varianti *anti-entropy* e *rumor mongering*. Il sistema qui presentato usa invece un **push one-hop a fanout fisso k con grpc** (Sezione IVB, indicato anche come *k-push*); i vicini sono scelti casualmente a ogni round, un modello neutro di una qualunque politica di selezione reale (prossimità di rete, similarità dei dati locali se disponibile, o un altro stato osservabile del nodo). I confronti che seguono si riferiscono alle scale testate in questo lavoro, $N \leq 8$ — il tetto imposto da AWS Learner Lab (Sezione IVF), non un limite intrinseco del protocollo.

Perché non flooding. Il motivo principale è il traffico: il numero di trasmissioni cresce con il grado medio dei vicini e, soprattutto, con il *time-to-live* — ogni hop aggiuntivo moltiplica i messaggi per il fattore di ramificazione della rete. Il caso limite $k = N - 1$ del nostro push one-hop è di fatto equivalente a un singolo round di flooding con *time-to-live* = 1: ogni worker raggiunge direttamente tutti i peer noti, senza bisogno di alcun inoltro successivo. La differenza pratica tra le due soluzioni emerge solo a *time-to-live* > 1, cioè quando servirebbe più di un hop per coprire l'intera rete. Su reti più grandi e sparse il traffico crescerebbe in modo moltiplicativo a ogni hop: resta quindi la soluzione più costosa delle tre a qualunque scala.

Il push one-hop equivale a N k-random-walk simultanei. Il singolo hop di trasporto coincide di fatto con il primo passo di un *k-random-walk* con *time-to-live* = 1. La differenza non è tanto il *time-to-live* quanto la molteplicità delle origini: un *k-random-walk* parte da *un solo* nodo con una query da propagare; il nostro sistema fa ripartire lo stesso meccanismo da *ogni* worker, ogni round, in parallelo.

Gossip epidemico. È un'alternativa legittima: il relay multi-hop propagherebbe gli aggiornamenti più rapidamente e più fedelmente all'originale, un vantaggio reale su reti molto più grandi di quelle testate, dove un k fisso non basterebbe più a coprire l'intera rete in pochi round. Il costo è il traffico (requisito del progetto) — ogni hop di relay moltiplica i messaggi per il fattore di ramificazione, mentre il push one-hop mantiene un costo per nodo indipendente da N — quindi un compromesso esplicito tra traffico e qualità/freschezza dell'informazione, da valutare caso per caso. La nostra soluzione, essendo applicata ad un contesto di machine learning federato con fed avg, comunque propaga ragionevolmente nel tempo l'informazione a tutta la rete, anche se hop è 1, in quanto i messaggi con i nuovi pesi hanno "memoria" anche dei vecchi aggiornamenti. Infatti il guadagno del relay è inoltre già in parte attenuato dal dominio applicativo: grazie a FedAvg, lo stato che un worker invia è già il risultato di aggregazioni con i round precedenti, quindi la propagazione multi-hop avviene comunque, implicitamente, nel tempo anziché nello spazio del singolo round. Il relay porterebbe anche un problema standard di deduplicazione — lo stesso messaggio potrebbe raggiungere lo stesso worker da percorsi indipendenti nello stesso round

— comunque risolvibile. Un'osservazione: quando il payload scambiato è l'intero stato di un modello e non un piccolo messaggio, il costo di un relay multi-hop cresce con il fattore di ramificazione a ogni hop, mentre l'aggregazione locale ottiene una propagazione senza traffico aggiuntivo dedicato al solo inoltro. In definitiva, abbiamo preferito restare entro i vincoli di traffico richiesti esplicitamente dalla traccia di progetto; resta un'osservazione interessante da approfondire in futuro come si comporterebbe il gossip epidemico in un contesto con molti più nodi, e come si confronterebbe con la soluzione qui proposta.

Verifica empirica. Il grafo one-hop raggiunge comunque la connettività completa alle scale qui testate: con $k = N - 1$ ogni worker contatta *tutti* i propri peer a ogni round, il limite superiore raggiungibile in un singolo hop. I risultati (Sezione VA, Sezione VIA) mostrano che questo da solo chiude la maggior parte del divario osservato a fanout basso (+7–8 punti di accuratezza global-test tra $k = 1$ e $k = N - 1$ a $N = 8$, spread quasi dimezzato); ma anche a $k = N - 1$ lo spread non scende mai sotto il 14% (Sezione VID). Poiché $k = N - 1$ è già la copertura massima raggiungibile in un hop, un relay multi-hop non avrebbe potuto ridurre ulteriormente questo residuo: la causa più probabile non è la portata del protocollo di comunicazione, ma l'assenza di un meccanismo esplicito di correzione del client drift (FedProx, SCAFFOLD, Sezione IID) — una limitazione indipendente dalla scelta tra one-hop e multi-hop, discussa in Sezione VID.

III. PROGETTAZIONE DELLA SOLUZIONE

A. Panoramica architetturale

Il sistema è composto da K nodi *worker* (con la stessa architettura del modello di ml e dati differenti privati) e da un *discovery server* (Fig. 1). Ogni worker esegue due thread indipendenti: un server gRPC che riceve i modelli inviati dai vicini (Fase C di un altro worker) e li accumula in un buffer condiviso, e un ciclo di training che, a ogni round, aggrega quanto ricevuto, esegue addestramento locale e invia i propri pesi a un sottoinsieme di vicini. Qualunque worker può avviare il proprio processo di training in autonomia.

Il discovery server espone quattro sole operazioni REST (/register, /deregister, /peers, /health), implementate in Flask invece che in gRPC: una scelta di semplicità, giustificata dal fatto che il suo traffico resta minimo rispetto ai messaggi gRPC scambiati tra worker — la lista dei peer è letta dai worker una sola volta all'avvio (o su richiesta, in caso di timeout verso un peer), contro gli $N \times k$ messaggi gRPC per round dedicati al trasporto dei pesi (Sezione V). Il discovery server mantiene in memoria una mappa *worker_id* → *indirizzo* e non riceve, memorizza o media mai un peso del modello. Il suo unico compito aggiuntivo è un *watchdog* che forza la terminazione del cluster se un worker non si deregistra correttamente entro un timeout — una funzione di coordinamento della terminazione, che ci è stata utile nel debug. Il raggio d'azione del registry è inoltre limitato nel tempo: la lista dei peer è letta una sola volta all'avvio (Sezione IVB) oppure quando si vuole riaggiornare

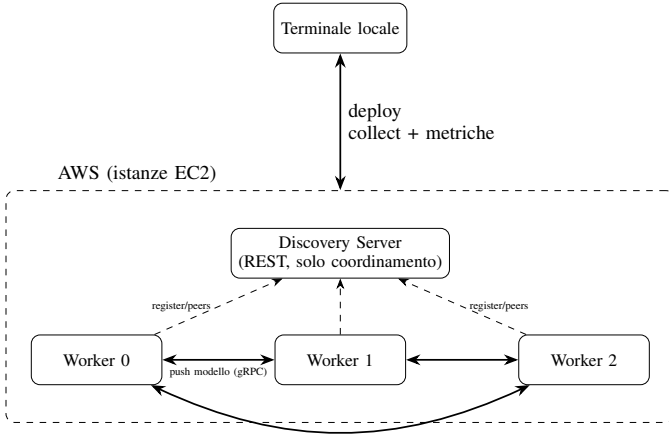


Fig. 1. Architettura logica: il discovery server gestisce solo la registrazione (linee tratteggiate REST); i modelli viaggiano esclusivamente peer-to-peer via gRPC (freccie continue). Il grafo tra worker è *potenziale*, non fisso: a ogni round ciascun worker sceglie casualmente k destinatari tra i peer disponibili, senza inoltro da parte di chi riceve (Sezione IIE). Il riquadro tratteggiato rappresenta le istanze AWS; il terminale locale lancia il deployment, raccoglie i risultati e calcola le metriche aggregate a fine training.

la cache in caso di timeout da parte dei worker. Un guasto del discovery server dopo l'avvio blocca solo l'ingresso di nuovi client, non gli scambi già in corso tra quelli attivi, che continuano a operare sulla propria cache locale.

B. Aggregazione decentralizzata

Ogni worker mantiene un buffer di aggregazione protetto da lock. Quando un messaggio arriva, il server gRPC non lo accoda: lo somma immediatamente, pesato per il numero di campioni del mittente, a un accumulatore già esistente. Il costo in memoria e in tempo di questa operazione è quindi $O(\text{dimensione del modello})$ e non $O(K \cdot \text{dimensione del modello})$ come sarebbe mantenendo una lista di tutti i modelli ricevuti: il buffer conserva sempre un solo modello accumulato, non uno per ciascun vicino da cui è arrivato un aggiornamento. A $N = 8$ con $k = N - 1 = 7$ (Sezione V), un worker può ricevere fino a 7 messaggi in un round: mantenerli distinti richiederebbe $7 \times 6,82 \approx 47,7$ MB, contro i $\approx 6,82$ MB costanti dell'accumulatore, a prescindere da quanti messaggi arrivino. All'inizio del round successivo, il thread di training applica FedAvg (Eq. 1) tra il proprio modello e la somma pesata accumulata, quindi svuota il buffer.

C. Motivazione delle scelte protocollari

La comunicazione tra worker usa gRPC su Protobuf anziché REST/JSON: la serializzazione binaria di un `state_dict` PyTorch produce un messaggio 2–3× più compatto rispetto alla codifica testuale equivalente, un risparmio non trascurabile quando ogni messaggio trasporta l'intero modello (Sezione V). La lista dei peer è letta dal discovery server una sola volta, all'avvio (Sezione IVA), e tenuta in una cache locale per l'intera durata del training: se il worker la reinterrogasse a ogni round, il traffico verso il discovery server crescerebbe linearmente con il numero di round anziché restare $O(1)$ per

l'intera run, vanificando in parte l'obiettivo di traffico ridotto che motiva anche la scelta del push one-hop a fanout fisso (Sezione IIE). La cache viene aggiornata solo lazily, cioè non periodicamente ma esclusivamente quando serve: ogni invio ha un timeout configurabile e, in caso di fallimento, il worker interroga di nuovo il discovery server e ritenta una sola volta con peer non ancora contattati in quel round — evitando sia blocchi su nodi caduti sia un sovraccarico di richieste al discovery server nei round in cui tutto funziona normalmente.

IV. DETTAGLI IMPLEMENTATIVI

A. Partizionamento non-i.i.d. del dataset

Il dataset FEMNIST scaricato in formato LEAF è partizionato per scrittore: gli scrittori sono raggruppati in blocchi contigui e ogni blocco è assegnato a un worker, cosicché ogni client riceva gli stili di scrittura e le distribuzioni di etichette di più utenti distinti, non di un singolo scrittore. Da ciascuna partizione locale è riservato un 10% per la validazione (usata per early stopping e come metrica di accuratezza riportata). Una frazione separata di scrittori del test set LEAF, mai assegnata a nessun worker, è riservata come *global test set* condiviso: valutare tutti i worker sullo stesso insieme di scrittori mai visti permette di misurare la convergenza *funzionale* del protocollo di comunicazione, non solo la vicinanza dei pesi. Va detto che un global test set di questo tipo non è detto sia disponibile in uno scenario reale: resta comunque un agglomerato di dati privati. In questo lavoro lo si è reso necessario per uno scopo puramente valutativo, misurare la convergenza funzionale del sistema durante la sperimentazione. Non è applicato alcun pre-processing o data augmentation: introdurre trasformazioni casuali (rotazioni, rumore) per worker altererebbe artificialmente l'eterogeneità naturale degli stili di scrittura, che è proprio l'oggetto di studio di un sistema FL non-i.i.d. E' chiaro come in uno scenario reale, i veri proprietari dei dati, potrebbero applicare tecniche di data augmentation localmente, col fine di combattere la reale natura non iid dei dati. Il sistema implementa anche un *local test set* opzionale (`local_test_set: true`): un'ulteriore frazione di campioni riservata all'interno della partizione di ciascun worker, dagli *stessi* scrittori già presenti in train/val di quel worker ma con campioni distinti, tenuti fuori dagli aggiornamenti del gradiente — misura la generalizzazione su campioni mai visti di scrittori già noti al worker, non su scrittori nuovi. Non è stato usato nella campagna sperimentale di questo lavoro (Sezione V): il global test set risponde già alla domanda più rilevante per un sistema FL P2P — la convergenza *funzionale tra worker* su scrittori ignoti a tutti — mentre il local test set misurerebbe una generalizzazione più ristretta, sugli stili di scrittura già visti da un singolo worker.

B. Ciclo di training a tre fasi

Ogni round di ogni worker è composto da tre fasi, cronometrate indipendentemente e riportate nella Sezione V.

La sincronizzazione avviene su un numero fisso di passi H , non su epoche: le partizioni non sono bilanciate tra worker (Sezione IVA), quindi un criterio "un round = un'epoca"

farebbe comunicare più spesso i worker con meno dati, rendendo la frequenza di comunicazione una conseguenza indiretta del partizionamento anziché un parametro controllato direttamente — a parità di H , tutti i worker comunicano alla stessa cadenza indipendentemente da quanti dati hanno (ma sempre in funzione dell’hardware).

La struttura è a tre livelli annidati, dal più piccolo al più grande. Il *minibatch* è l’unità elementare: `batch_size=32` campioni, un passo di SGD (`train_step`). L’*epoca* è un giro completo sulla partizione locale del worker — l’ultimo minibatch incompleto viene scartato (`drop_last=True`), per non avere batch di dimensione variabile che destabilizzerebbero le statistiche di Batch Normalization. Il *round* — l’unità su cui il sistema sincronizza — è H minibatch consecutivi: gli *inner step* di DiLoCo (Sezione IIB) sono letteralmente H ripetizioni di un minibatch, non H epoche, e possono quindi coprire una piccola frazione di un’epoca, un’epoca esatta, o più epoche a seconda di come $H \times 32$ si confronta con la dimensione della partizione — il caso concreto è discusso in Fase B qui sotto.

- **Fase A — aggregazione e valutazione.** Il worker applica FedAvg (Eq. 1) tra i propri pesi e la somma pesata accumulata nel buffer di ricezione, quindi valuta il modello risultante sul proprio validation set e, se abilitato, sul global test set. La Fase A include quindi non solo la media dei pesi (costo trascurabile) ma anche l’inferenza completa su questi due insiemi di dati.
- **Fase B — addestramento locale.** Il worker esegue H passi di SGD (AdamW, batch size 32) su un iteratore che ricicla il proprio training set indefinitamente, cosicché ogni round processi esattamente $H \times \text{batch_size}$ campioni indipendentemente dalla dimensione della partizione locale. L’iteratore è un generatore continuo (`while True: yield from loader`), creato una sola volta prima del ciclo dei round: se un’epoca termina durante i passi H di un round, il generatore prosegue trasparentemente nell’epoca successiva con un nuovo shuffle, senza che il confine di round interferisca. Nelle configurazioni qui testate un’epoca non si esaurisce comunque mai entro un singolo round, quindi non si verificano letture ripetute dello stesso campione *all’interno* di un round: anche al valore massimo $H = 1000$ (32 000 campioni per round) si resta ben sotto la partizione più piccola tra quelle testate ($\sim 158\,500$ campioni a $N = 3$).
- **Fase C — push one-hop.** Il worker seleziona $\min(k, |\text{peer disponibili}|)$ vicini *casualmente e in modo indipendente a ogni round* (non c’è memoria della selezione dei round precedenti) dalla propria cache di peer, e invia loro uno snapshot dei pesi correnti via gRPC con timeout configurabile. La selezione casuale non è un vincolo del protocollo ma un modello neutro di qualunque politica di selezione dei vicini si volesse adottare in pratica — prossimità di rete, similarità tra i dati locali del worker, o un qualsiasi altro stato osservabile del nodo; ha inoltre il vantaggio di non richiedere alcuna

TABLE I
CONTEGGIO DEI PARAMETRI DELLA CNN.

Componente	Parametri
Blocco conv. 1 (1→32, 32→32 canali)	9 696
Blocco conv. 2 (32→64, 64→64 canali)	55 680
Classificatore (3136→512→62, con BN)	1 638 974
Totale	1 704 350

conoscenza della topologia sottostante. Se un peer non risponde entro il timeout, il worker aggiorna la propria cache interrogando di nuovo il discovery server — al più una volta per round, solo in caso di fallimento — e tenta un singolo invio verso un peer sostitutivo non ancora contattato in quel round. Se anche questo secondo tentativo fallisce, il worker non ritenta ulteriormente né richiama di nuovo il registry in quel round: prosegue senza bloccarsi sul nodo caduto. La scelta mantiene il traffico verso il discovery server limitato a una sola chiamata per round anche nel caso peggiore, accettando che un peer possa restare non contattato per quel round.

Un controllo di *staleness* lato ricevente scarta i messaggi il cui round di provenienza è più vecchio di `max_staleness` round rispetto al round corrente del ricevente, evitando che aggiornamenti molto obsoleti degradino la convergenza. L’early stopping (pazienza configurabile) è una decisione puramente locale basata sulla validation loss del singolo worker: un worker può terminare il proprio training mentre altri continuano, riducendo nel tempo il numero di vicini disponibili per chi prosegue. Lo stesso criterio determina anche quale checkpoint (`model_best.pt`) viene salvato. Usare la performance sul global test set per scegliere quale round tenere costituirebbe una forma di *data leakage*, gonfiando artificialmente il risultato riportato verso il round che per puro rumore statistico va meglio su quel set specifico. Il global test set resta così una valutazione mai usata per decisioni, solo per misurare a posteriori.

C. Modello

Il modello (Tabella I) è una CNN con due blocchi convoluzionali (32 e 64 canali, batch normalization, dropout spaziale) e un classificatore fully-connected, per un totale di 1 704 350 parametri float32 — circa 6,82 MB per snapshot serializzato, la quantità di dati effettivamente trasmessa a ogni push.

La progressione di canali $1 \rightarrow 32 \rightarrow 64$ approssima una conservazione del volume di informazione a ogni dimezzamento della risoluzione spaziale ($28 \rightarrow 14 \rightarrow 7$); il kernel 3×3 in cascata ottiene il campo ricettivo di una 5×5 con meno parametri, e il same padding preserva la dimensione spaziale fino al pooling. Un’architettura più pesante (es. ResNet-18, $\sim 11\text{M}$ parametri, $\sim 6 \times$ questo modello) sarebbe sovradimensionata a questa risoluzione: i tratti di un carattere scritto a mano sono già ben rappresentati da 2–3 layer convolutivi, e la profondità o le connessioni residuali aggiuntive tipiche

di ResNet servono a catturare gerarchie di feature molto più complesse (immagini naturali, centinaia di pixel per lato) che qui si suppone non servono. In un sistema FL dove il modello viene interamente ritrasmesso a ogni push, la dimensione è fondamentale: non solo come capacità, ma come costo di comunicazione diretto. Il modello qui usato è quindi un compromesso deliberato tra capacità sufficiente per il compito e leggerezza per il contesto FL.

Batch normalization stabilizza il training; nel contesto federato solo γ, β sono aggregati da FedAvg, le statistiche correnti restano locali — limitazione discussa in Sezione VI. Dropout spaziale ($p = 0,25$) e dropout standard ($p = 0,5$ sul layer più esposto a overfitting con $\sim 1,6M$ parametri) regolarizzano le partizioni locali. L’inizializzazione dei pesi usa il default di PyTorch, Kaiming uniform per Conv2d e Linear [6]. Il modello è addestrato con AdamW.

D. Tolleranza ai guasti

Il sistema include meccanismi opzionali di fault injection configurabili (probabilità di crash simulato via `sys.exit` e di drop silenzioso di un messaggio), implementati ma non esercitati in nessuno dei 16 run riportati (`crash_probability` e `drop_probability` sono nulle in tutta la campagna sperimentale, Sezione V): la loro validazione empirica non è stata effettuata per mancanza di tempo e resta lavoro futuro. Il comportamento previsto dal codice è il seguente: un crash simulato interrompe il round corrente con `sys.exit`, che attraversa comunque il blocco di terminazione che deregistra il worker — un comportamento più vicino a un arresto pulito che a un crash reale; gli altri worker non vengono bloccati e si accorgono dell’assenza del peer al primo fallimento di invio verso di esso, aggiornando la propria cache. Solo un segnale SIGKILL, non intercettabile, produce un’uscita realmente non pulita, che lascia una entry non valida nel discovery server fino al successivo fallimento di invio — limitazione discussa nella Sezione VI.

E. Motivazione degli iperparametri di training

Il gradient clipping ($\|\nabla\| \leq 1$) limita la deriva dei pesi tra aggregazioni; il label smoothing ($\epsilon = 0,1$ [8]) attenua l’eccesso di confidenza sulle classi ambigue di FEMNIST (es. 0/O, 1/I/I).

L’ottimizzatore è AdamW [7] anziché SGD o Adam semplice, per due motivi. Primo, AdamW disaccoppia il weight decay dall’aggiornamento basato sul gradiente adattivo: in Adam classico, la regolarizzazione L2 interagisce con i learning rate adattivi per-parametro in modo che ne attenua l’effetto in modo non uniforme tra i parametri; AdamW applica il weight decay direttamente ai pesi, indipendentemente dalla scala del gradiente, un comportamento più prevedibile per un iperparametro (`weight_decay`) che qui resta comunque fisso. Secondo, un ottimizzatore adattivo per-parametro richiede meno tuning del learning rate di SGD per convergere in modo ragionevole — utile in questo lavoro, dove il budget sperimentale è speso sui tre parametri di sistema (H, k, N) e non è disponibile per una ricerca separata del learning rate ottimale

per ciascuna delle 16 configurazioni; è inoltre coerente con l’eterogeneità non-i.i.d. tra worker, dove ciascuno ottimizza una loss locale diversa: un tasso adattivo per-parametro si adegua meglio a superfici di loss locali diverse rispetto a un tasso fisso unico calibrato per tutte.

AdamW con learning rate 10^{-3} e batch size 32 sono *fissi* e legati dalla *linear scaling rule*: variarli insieme avrebbe introdotto un confondimento non separabile col numero limitato di run. Nessuna cross-validation: selezione manuale, un run per configurazione.

I parametri seguono quindi tre categorie. *Iperparametri ML fissi*: learning rate, batch size, dropout, label smoothing, gradient clipping, architettura della CNN — mai oggetto della griglia. *Parametri di sistema variati* (Sezione V): $H \in \{100, 500, 1000\}$ (Blocco A, $N = 3$ fisso; 500 è il valore di riferimento di DiLoCo [1]), il fanout k e $N \in \{3, 5, 8\}$ (Blocchi B e C) — i soli tre parametri di cui questo lavoro misura l’effetto comparativo. Il limite superiore $N = 8$ non è una scelta di progetto ma un vincolo della piattaforma di test: AWS Learner Lab consente al massimo 9 istanze EC2 concorrenti per account, una delle quali è riservata al discovery server, lasciando $N \leq 8$ worker disponibili (Sezione IVF). *Parametri strutturali fissi*: FedAvg, `max_staleness=10`, fault injection disattivata.

F. Piattaforma software

Il sistema è implementato in Python 3.11 con PyTorch (modello e training), gRPC/Protobuf (comunicazione push tra worker), Flask (discovery server), containerizzato con Docker e orchestrato con Docker Compose in locale. Il deployment su AWS usa Terraform per provisionare un’istanza EC2 per worker più una per il discovery server (modalità multi-istanza, rete reale tra macchine fisicamente separate), con build dell’immagine Docker in parallelo su ciascuna istanza. L’account didattico AWS Learner Lab usato per i test impone un tetto di 9 istanze EC2 concorrenti: una è occupata dal discovery server, per cui il numero massimo di worker testabile è $N = 8$ — il limite superiore della griglia sperimentale (Sezione V) riflette questo vincolo della piattaforma, non un limite del protocollo, che non assume alcun tetto su N .

Le immagini Docker sono costruite sfruttando il *layer caching*: ogni istruzione del Dockerfile produce un layer immutabile riutilizzato se il suo hash coincide con la build precedente, e l’invalidazione è a cascata (un layer modificato invalida tutti quelli successivi). Il Dockerfile del worker copia e compila `proto/gossip.proto` *prima* di copiare il resto del sorgente Python: il layer più costoso, l’installazione di PyTorch ($\sim 750MB$), viene rieseguito solo se cambia `requirements.worker.txt`, non ad ogni modifica al codice — una modifica al codice invalida solo l’ultimo layer, riducendo la rebuild da minuti a pochi secondi. Tutti i worker di un deployment condividono inoltre un’unica immagine (`docker compose up --build` la costruisce una sola volta): i layer read-only, incluso quello di PyTorch, sono condivisi su disco tra tutti i container, che aggiungono solo un sottile layer scrivibile per i propri file di runtime.

Il sistema non include una suite di test automatizzati (unit o integration test); la sua validazione è di tipo empirico, tramite esecuzione end-to-end sulle 16 configurazioni sperimentali di Sezione V, con verifica incrociata delle metriche registrate (conteggio dei messaggi scambiati, tempi di fase, coerenza tra round completati e pazienza configurata) contro i log grezzi prodotti da ciascun worker.

G. Vincolo di RAM e caricamento del dataset

Il caricamento del dataset non è *lazy*: `FEMNISTDataset.__init__` converte l'intero split di training in tensori tenuti in memoria all'avvio, una scelta di semplicità che però si scontra con il vincolo di RAM delle istanze `t3.small` usate su AWS (2 GB, Sezione V). Il formato LEAF serializza i pixel come numeri in JSON: durante il parsing, ogni valore diventa un oggetto `float` Python, che occupa ~ 24 byte per via dell'overhead dell'interprete, contro i 4 byte di un `float32` compatto. Con $N = 8$ worker ($\sim 100\,000$ immagini ciascuno, 784 pixel per immagine) il picco di RAM durante il solo parsing JSON era $100\,000 \times 784 \times 24 \text{ byte} \approx 1.9 \text{ GB}$ per singolo worker; con $N = 3$ ($\sim 267\,000$ immagini ciascuno) saliva a $\sim 4.6 \text{ GB}$ — ben oltre i 2 GB disponibili. Il sintomo era un OOM kill (SIGKILL) su worker scelti a caso durante l'avvio, con il cluster bloccato a $N-1$ worker registrati su N attesi.

La correzione separa il parsing dal caricamento a runtime. `split_dataset.py` esegue ora una terza passata che rilegge il JSON già scritto per ciascun worker e lo converte, una sola volta, in array NumPy tipizzati (`float32 + int64`), salvati come `data.npy/labels.npy` accanto al JSON originale. `dataset.py` carica questi binari con `np.load` quando presenti, con fallback al JSON per gli split generati prima della modifica: niente più oggetti `float` Python intermedi, il picco di RAM scende alla sola dimensione dell'array ($\sim 310 \text{ MB}$ a $N = 8$, $\sim 830 \text{ MB}$ a $N = 3$), e `torch.from_numpy` condivide la memoria con l'array NumPy senza copie aggiuntive. Il *global test set*, letto in sola lettura da tutti i worker, usa inoltre `np.load(mmap_mode='c')`: nelle modalità in cui più worker condividono la stessa macchina (locale, single-EC2) il sistema operativo mappa le stesse pagine fisiche per tutti i processi che aprono il file, azzerando la duplicazione che si avrebbe altrimenti per worker; in modalità multi-istanza, dove ogni EC2 ospita un solo worker, questo beneficio di condivisione non si applica, ma `mmap` resta comunque corretto ed evita il parsing JSON. Le partizioni per-worker non usano `mmap`: ogni worker legge un file diverso, quindi nessuna condivisione tra processi è possibile, e un array contiguo in RAM resta preferibile per l'accesso casuale del training loop con shuffle.

H. Riutilizzabilità

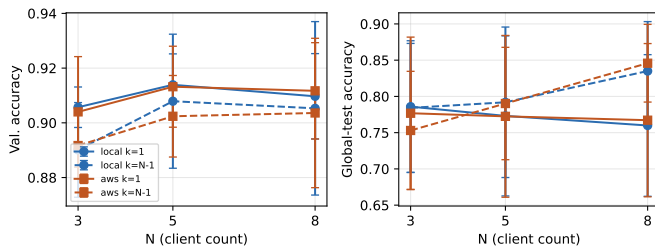
Il sistema è progettato per non dipendere dalle specificità di FEMNIST. Il partizionamento non-i.i.d. e l'aggregazione pesata per numero di campioni (Eq. 1) non assumono client bilanciati o distribuzioni simili tra loro: l'eterogeneità è il caso

di base gestito dal progetto, non un'eccezione da accomodare — un dataset LEAF diverso (es. Shakespeare, Sentiment140) o una partizione non-i.i.d. più estrema richiederebbero solo un nuovo caricatore dati (`core/dataset.py`), non una riprogettazione della logica di training o di comunicazione. L'unica dipendenza strutturale esterna è il discovery server, che espone quattro soli endpoint REST: adattare il sistema a un'altra infrastruttura (altro cloud provider, rete locale, un mix di dispositivi edge) richiede solo di ripuntare l'URL del registro, non di modificare il protocollo di aggregazione. Il protocollo di comunicazione stesso è agnostico rispetto all'architettura del modello — serializza un intero `state_dict` PyTorch senza logica specifica per `FEMNISTModel` — per cui sostituire la rete neurale richiede di cambiare un solo punto di istanziazione, non il livello di rete. Infine, il numero di client, il fanout k e tutti gli iperparametri sono parametrizzati in un unico file di configurazione: adattare il sistema a una nuova scala non richiede modifiche al codice, come dimostrato direttamente dai run a $N = 3, 5, 8$ di Sezione V. La scelta di gRPC su Protobuf è motivata anche da questo obiettivo di riutilizzabilità: lo schema `gossip.proto` (Sezione IIIC) è indipendente dal linguaggio e gRPC genera client/server per numerosi linguaggi, per cui un worker non Python potrebbe in linea di principio interoperare in un contesto eterogeneo. Questo vale però solo a livello di trasporto e metadati: il campo `weights` resta un payload opaco serializzato con `torch.save`, specifico di Python — l'interoperabilità end-to-end richiederebbe un formato di tensori neutro (es. `safetensors`).

V. RISULTATI

Sono stati eseguiti 16 esperimenti su una griglia comune di configurazioni (numero di client $N \in \{3, 5, 8\}$, fanout k , passi locali H), metà in esecuzione locale (container Docker sulla stessa macchina) e metà su AWS EC2 in modalità multi-istanza (un'istanza `t3.small` per worker più una `t3.micro` per il discovery server, collegate da rete reale). In entrambi gli ambienti l'esecuzione è avvenuta su CPU. Per ogni configurazione sono riportati: l'accuratezza di validazione media tra i worker, l'accuratezza sul global test set (calcolata al round del checkpoint migliore di ciascun worker — la metrica indicata come primaria dallo strumento di analisi, più stabile della misura al solo round finale) con il relativo *spread* (differenza massimo-minimo tra worker, in punti percentuali), la distanza L2 media tra i pesi finali dei worker, il numero totale di messaggi scambiati e il wall-clock di sistema. La Tabella II riporta tutti e 16 i run.

Questa griglia non è un fattoriale completo su N , k e H : ammettendo k fino a $N-1$ per ciascun $N \in \{3, 5, 8\}$ e 3 valori di H , un fattoriale completo richiederebbe 39 configurazioni per ambiente (78 in totale) — con run AWS di oltre 4 ore ciascuno (Tabella II), settimane di esecuzione. È stato quindi adottato un disegno sequenziale a blocchi, un fattore alla volta: il Blocco A (3 run, $N = 3$, $k = 1$) isola l'effetto di $H \in \{100, 500, 1000\}$, individuando $H = 1000$ come migliore; il Blocco B (1 run) isola un piccolo aumento di k ; il Blocco C (4 run, $H = 1000$) incrocia $N \in \{5, 8\}$ con $k \in \{1, N-1\}$,



i due estremi del fanout, isolando l'effetto congiunto di scala e grado di connettività — il compromesso centrale di questo lavoro. Il costo: un'assunzione non verificata, che l' H migliore a $N = 3$ resti valido a $N = 5, 8$.

A. Scalabilità: accuratezza e tempo vs. N

La Fig. 2 mostra accuratezza di validazione e global-test al variare di N , per $H = 1000$ fissato, confrontando $k = 1$ (fanout minimo) con $k = N - 1$ (broadcast completo a singolo hop). L'accuratezza di validazione resta stabile tra l'89% e il 91% per ogni N testato, con variazioni contenute entro un punto percentuale tra $k = 1$ e $k = N - 1$: il sistema scala senza perdita di qualità sul dato visto localmente. Sul global test set l'effetto di k è più marcato: a $N = 8$, passare da $k = 1$ a $k = N - 1$ porta l'accuratezza global-test dal 76,0% all'83,5% in locale e dal 76,7% all'84,6% su AWS — un guadagno di 7–8 punti percentuali attribuibile a un numero maggiore di aggiornamenti ricevuti per round.

B. Overhead di comunicazione

Ogni messaggio push trasporta l'intero `state_dict` del modello (1 704 350 parametri float32, $\approx 6,82$ MB). Il traffico totale stimato (Fig. 3) cresce con N e, soprattutto, con k : a $N = 8$, passare da $k = 1$ a $k = N - 1$ porta il numero di messaggi scambiati da 295 a 1863 in locale ($6,3\times$) e da 267 a 1296 su AWS ($4,9\times$). Questo è il costo diretto, e prevedibile, della scelta di k : l'aumento di accuratezza global-test osservato con k alto (Fig. 2) va quindi letto come un compromesso esplicito comunicazione/qualità, non come un miglioramento gratuito.

C. Dove viene speso il tempo

La Fig. 4 scompone la durata media di un round nelle tre fasi definite in Sezione IVB. In locale la Fase A (merge e doppia valutazione) dura 5–13 s, la Fase B (H passi locali) 0,3–6,9 s in funzione di H , la Fase C (invio push) resta sotto i 340 ms anche con $k = N - 1$. Su AWS la Fase A si stabilizza attorno a 105–130 s indipendentemente da N e da H — un valore pressoché costante compatibile con un collo di bottiglia sul throughput di inferenza delle istanze `t3.small` (CPU burstable condivisa) piuttosto che con un effetto del workload. La Fase C, invece, resta nello stesso ordine di grandezza del caso locale (42–292 ms) anche attraverso rete reale tra istanze EC2 separate: l’overhead di comunicazione del protocollo di push non è la causa del rallentamento osservato su AWS.

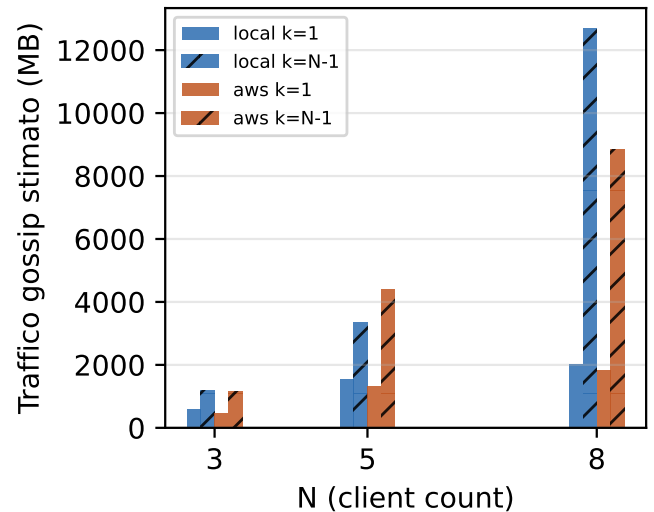


Fig. 3. Traffico totale stimato (messaggi $\times$ 6,82 MB) per configurazione.

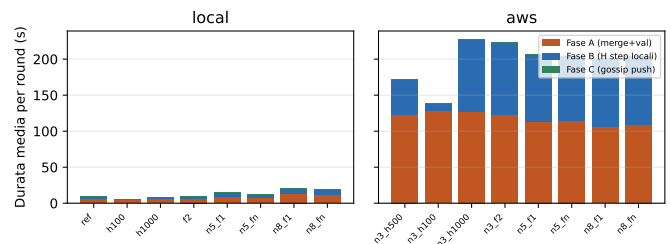


Fig. 4. Durata media delle fasi A, B, C per round, locale vs. AWS.

D. Locale vs. AWS multi-istanza

Per le 8 configurazioni condivise dai due ambienti, la Fig. 5 confronta wall-clock di sistema e spread global-test. Il wall-clock su AWS è 6–30× superiore a quello locale per la stessa configurazione (dominato dalla Fase A, come discusso sopra), mentre lo spread global-test resta nella stessa banda (14–25%) in entrambi gli ambienti, senza un vantaggio sistematico dell’uno sull’altro: la qualità della convergenza funzionale non dipende in modo evidente dall’ambiente di esecuzione, mentre il tempo di completamento sì.

VI. DISCUSSIONE

Due osservazioni di sintesi guidano l’interpretazione dei risultati. Sulle prestazioni: la validation accuracy resta piatta (89–91%) nelle configurazioni con $H \geq 500$ perché riflette soprattutto l’adattamento di ciascun worker ai propri dati locali dopo la Fase B, un segnale poco sensibile al protocollo di comunicazione; scende invece a 87–88% nei due run a $H = 100$ (locale e AWS, Tabella II), dove ogni round processa un decimo dei campioni prima dell’aggregazione. Global-test accuracy e spread, invece, variano sistematicamente con k e N perché misurano la reale convergenza collaborativa e non il solo adattamento locale — sono quindi le metriche rilevanti per giudicare la strategia di distribuzione, discussa

TABLE II

RISULTATI PER TUTTE LE 16 CONFIGURAZIONI SPERIMENTALI (k =FANOUT, H =PASSI LOCALI; SPREAD E L2 SUL GLOBAL TEST SET — SEZ. V).

Config	Ambiente	N	k	H	Val. acc.	Global-test acc.	Spread	L2	Msg inviati	Wall-clock
ref	locale	3	1	500	0.898 ± 0.010	0.769 ± 0.101	19.1%	102.9	88	5.2 min
h100	locale	3	1	100	0.874 ± 0.024	0.760 ± 0.126	24.9%	235.4	66	5.0 min
h1000	locale	3	1	1000	0.906 ± 0.007	0.786 ± 0.091	17.9%	184.3	87	8.7 min
f2	locale	3	2	1000	0.890 ± 0.017	0.784 ± 0.089	15.4%	107.7	172	8.1 min
n5_fn	locale	5	1	1000	0.914 ± 0.011	0.773 ± 0.110	21.2%	90.7	228	15.8 min
n5_fn	locale	5	4	1000	0.908 ± 0.025	0.792 ± 0.104	21.8%	117.0	491	12.6 min
n8_fn	locale	8	1	1000	0.910 ± 0.016	0.760 ± 0.098	22.4%	174.0	295	19.9 min
n8_fn	locale	8	7	1000	0.905 ± 0.032	0.835 ± 0.068	15.2%	94.7	1863	24.0 min
n3_h500	AWS	3	1	500	0.901 ± 0.020	0.756 ± 0.119	21.1%	137.9	85	158.1 min
n3_h100	AWS	3	1	100	0.883 ± 0.019	0.734 ± 0.116	20.3%	75.9	119	133.1 min
n3_h1000	AWS	3	1	1000	0.904 ± 0.020	0.777 ± 0.105	19.5%	138.0	67	150.3 min
n3_f2	AWS	3	2	1000	0.892 ± 0.012	0.753 ± 0.082	14.3%	88.9	171	160.8 min
n5_fn	AWS	5	1	1000	0.913 ± 0.015	0.773 ± 0.112	20.8%	88.3	193	144.8 min
n5_fn	AWS	5	4	1000	0.902 ± 0.015	0.790 ± 0.078	18.3%	105.7	646	247.6 min
n8_fn	AWS	8	1	1000	0.912 ± 0.018	0.767 ± 0.106	22.1%	93.9	267	194.1 min
n8_fn	AWS	8	7	1000	0.904 ± 0.027	0.846 ± 0.054	14.0%	57.1	1296	142.9 min

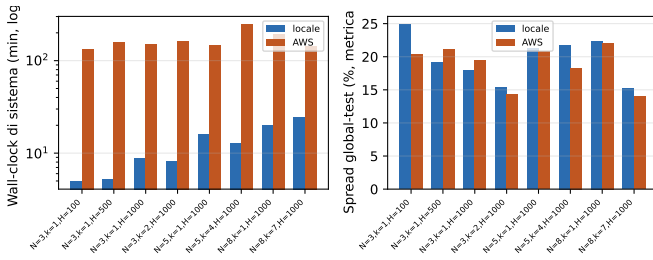


Fig. 5. Wall-clock di sistema (scala log) e spread di accuratezza global-test, per le 8 configurazioni condivise da locale e AWS.

di seguito. Sui tempi: il wall-clock è dominato dal calcolo locale (Fase A/B) e non dalla comunicazione (Fase C), come discusso più sotto — l’ambiente di esecuzione incide più della configurazione stessa.

A. Il fanout k come compromesso esplicito

A $N = 5$ e $N = 8$ (locale e AWS), un k più alto porta più aggiornamenti per round e un’accuratezza global-test più alta a parità di H — coerente con un mixing più denso che riduce la divergenza verso soluzioni funzionalmente diverse. A $N = 3$ l’effetto è piatto o leggermente negativo in entrambi gli ambienti: con solo due vicini possibili, $k = 1 \rightarrow 2$ non cambia qualitativamente la topologia, e il rumore tra worker (tre soli campioni) è dello stesso ordine dell’effetto misurato. Il guadagno non è gratuito: il traffico cresce nella stessa proporzione di k (Sez. VB), per cui k va sempre scelto rispetto al budget di banda, non solo all’accuratezza target.

B. Comunicazione non è il collo di bottiglia

Anche su rete reale tra istanze EC2 separate, la Fase C resta dell’ordine delle centinaia di millisecondi, mentre il rallentamento di oltre un ordine di grandezza su AWS è concentrato nella Fase A (Sez. VC), compatibile con un limite di CPU burstable delle `t3.small` più che con un costo di rete. Questo supporta la scelta di comunicare snapshot completi a bassa frequenza invece di aggiornamenti più frequenti e piccoli: il costo di calcolo locale domina comunque quello

di comunicazione, per cui ridurre ulteriormente la frequenza di comunicazione non avrebbe portato benefici proporzionali sul tempo totale.

C. Nota metodologica sulla metrica di convergenza funzionale

Lo strumento di analisi calcola lo spread sul global test set in due varianti: al round finale di ciascun worker (etichettata riferimento rumoroso) e al round del checkpoint migliore (metrica dichiarata primaria). Le due possono discordare sensibilmente: per due configurazioni a $N = 3$, la variante riferimento suggeriva uno spread contenuto (4,5–4,8%), mentre la primaria mostra per le stesse run 19,5–21,1%, in linea con le altre 14. Si è scelto di riportare sempre e solo la metrica primaria (Tabella II), per non presentare come rappresentativo un valore isolato ottenuto con la metrica esplicitamente definita rumorosa.

D. Limitazioni

- **Convergenza funzionale incompleta.** In nessuna configurazione lo spread sul global test set scende sotto il 14%, nemmeno a connettività one-hop massima ($k = N - 1$, Sezione IIE): il push one-hop riduce ma non elimina la divergenza tra worker nel budget testato. I dati grezzi mostrano anche oscillazioni round-su-round marcate (in un caso, -28 punti percentuali tra checkpoint migliore e round finale di un worker), sintomo di instabilità dopo un merge FedAvg con vicini divergenti.
- **Singola replica per configurazione.** Nessun run è ripetuto con seed diverso: la deviazione standard riportata è tra worker dello stesso run, non tra run della stessa configurazione, e non separa con certezza l’effetto di un iperparametro dalla variabilità intrinseca.
- **Confondimento in una configurazione AWS.** `n8_fn` su AWS usa `early_stopping_patience=7` anziché 10 come tutti gli altri, per un errore di configurazione: il confronto diretto a $N = 8$ va interpretato con cautela.
- **Peer contattati sotto il target.** Il numero medio di vicini contattati può scendere sotto k quando altri worker terminano in anticipo e si deregistrano, riducendo il pool

disponibile — effetto emergente dell’asincronia, non un difetto della selezione.

- **Assenza di heartbeat.** Un peer caduto è rilevato solo al primo fallimento di invio; un crash non pulito (SIGKILL, OOM) lascia una entry non valida nel discovery server fino a quel momento. Scelta deliberata: un heartbeat periodico introdurrebbe traffico crescente con N , in contrasto con l’obiettivo di traffico ridotto che guida anche cache dei peer e k -push; costo non giustificato per un caso d’uso fuori dal perimetro sperimentale di questo lavoro.
- **Statistiche di Batch Normalization non aggregate.** Solo scala e bias sono mediati da FedAvg; media e varianza correnti restano locali, nota causa di disallineamento post-aggregazione in letteratura [4].
- **Nessuna metrica per classe.** `validate()` (Sezione IV) calcola solo loss aggregata e accuratezza top-1; non vengono salvate predizioni, logit o una confusion matrix. FEMNIST ha 62 classi con frequenze naturalmente sbilanciate (cifre vs. lettere, maiuscole vs. minuscole), e con partizionamento non-i.i.d. per scrittore alcuni worker vedono pochi esempi di certe classi: un’accuratezza aggregata alta può quindi mascherare una scarsa accuratezza sulle classi rare. Il dato non è recuperabile a posteriori sulle 16 run già eseguite: `model_best.pt` non viene archiviato da `save_experiment.py` — è cancellato dalla working directory subito dopo la copia delle metriche, per lasciare pulito l’ambiente alla run successiva — per cui un’analisi per classe richiederebbe di ripetere almeno una configurazione.

E. Confronto con la letteratura

FedAvg centralizzato su FEMNIST è tipicamente riportato attorno al 77–80% di accuratezza di test [2]. Le configurazioni qui testate raggiungono un’accuratezza di validazione più alta (87–91%) ma global-test più bassa (75–85%): confronto non del tutto omogeneo, poiché la validazione avviene su campioni degli stessi scrittori visti in training, mentre il global test contiene scrittori mai visti — un compito di generalizzazione più severo e più vicino allo scenario reale.

VII. CONCLUSIONI

Questo lavoro ha presentato un sistema di Federated Learning decentralizzato peer-to-peer, in cui FedAvg è eseguito tramite aggregazione online su pesi scambiati con un protocollo di comunicazione push one-hop a fanout fisso k , senza alcun aggregatore centrale per l’intera durata dell’addestramento, e in cui la centralizzazione è confinata alla sola discovery dei client. Il sistema è stato valutato su FEMNIST (LEAF) con partizionamento non-i.i.d. per scrittore, in 16 configurazioni tra esecuzione locale e deployment reale su AWS multi-istanza, raggiungendo un’accuratezza di validazione dell’87–91% e mostrando che, nel regime testato, il costo di comunicazione del protocollo resta trascurabile rispetto al costo del calcolo locale, anche su rete reale. L’analisi ha inoltre evidenziato, in modo trasparente, i limiti

della convergenza funzionale ottenuta e le condizioni sperimentali (singola replica, un confondimento di configurazione) sotto cui i risultati vanno interpretati.

Tra le 16 configurazioni testate, $N = 8$, $k = N-1 = 7$, $H = 1000$ emerge come quella con la miglior convergenza funzionale: la più alta accuratezza sul global test set e il più basso spread tra worker, in entrambi gli ambienti indipendentemente (83,5%/15,2% in locale, 84,6%/14,0% su AWS) — l’unico risultato di questo lavoro confermato in modo identico da entrambe le campagne sperimentali, e quindi la raccomandazione più solida che se ne possa trarre. Ha però anche il volume di traffico più alto di tutta la campagna (Tabella II): resta una raccomandazione condizionata a un budget di banda ampio, non un ottimo assoluto. Se il traffico è vincolante, $N = 5$, $k = N-1 = 4$, $H = 1000$ offre un’accuratezza global-test e uno spread solo leggermente peggiori a una frazione del volume di comunicazione. In entrambi i casi, $H = 1000$ resta il valore di passi locali raccomandato, l’unico iperparametro ML esplicitamente confrontato in questo lavoro (Sezione V).

Sviluppi futuri naturali includono: l’introduzione di una strategia di aggregazione alternativa a FedAvg (ad es. FedProx) per mitigare esplicitamente il client drift osservato; una politica di k adattivo che tenga conto del numero di peer ancora attivi; e la ripetizione delle configurazioni con seed multipli per quantificare rigorosamente la varianza run-to-run qui non misurata.

Va inoltre osservato che il sistema non prevede alcun meccanismo di *restart* automatico, né per i worker né per il discovery server: un crash richiede un intervento manuale, o un orchestratore esterno non implementato in questo lavoro (es. una restart policy Docker), per rilanciare il processo — i meccanismi seguenti presuppongono che il riavvio avvenga, non lo innescano. Due estensioni indirizzerebbero più a fondo la tolleranza ai guasti, oggi solo parzialmente coperta (Sezione VID). Per il rientro dopo un crash, il checkpoint locale già salvato a ogni miglioramento della validation loss potrebbe includere il numero di round: un worker riavviato si ri-registrerebbe con lo stesso identificativo e riprenderebbe da lì invece che da zero, con lo staleness check esistente ad assorbire il disallineamento rispetto agli altri. Per l’ingresso live di un client nuovo — il discovery server accetta già registrazioni in qualunque momento, ma oggi si partirebbe da pesi casuali — una RPC di *pull* accanto all’attuale *push* permetterebbe di richiedere una volta i pesi correnti a un peer attivo come inizializzazione. Questo risolverebbe però solo il problema dei pesi, non quello dei dati: il partizionamento (Sezione IVA) è precalcolato per un numero fisso di worker prima del deployment, quindi un nodo davvero nuovo non avrebbe una partizione propria — servirebbe riservare in anticipo dati per client futuri (spreco se non usati) o un re-sharding a runtime, più complesso e in tensione con il principio di non spostare mai dati grezzi tra nodi.

REFERENCES

- [1] A. Douillard, Q. Feng, A. A. Rusu, R. Chhaparia, Y. Donchev, A. Kuncoro, M. Ranzato, A. Szlam, and J. Shen, “Diloco: Distributed low-communication training of language models,” *arXiv preprint arXiv:2311.08105*, 2023.
- [2] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, “Federated optimization in heterogeneous networks,” *Proceedings of Machine Learning and Systems (MLSys)*, vol. 2, pp. 429–450, 2020.
- [3] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, “Scaffold: Stochastic controlled averaging for federated learning,” in *International Conference on Machine Learning (ICML)*, 2020, pp. 5132–5143.
- [4] X. Li, M. Jiang, X. Zhang, M. Kamp, and Q. Dou, “Fedbn: Federated learning on non-iid features via local batch normalization,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [5] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, “Delving deep into rectifiers: Surpassing human-level performance on imagenet classification,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 1026–1034.
- [7] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [8] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the inception architecture for computer vision,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2818–2826.